Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «АПО ЭВМиС»

Тема: «Архитектура и программирование сопроцессора»

Целочисленные и вещественные команды

Выполнил:
Студент 4-го курса
Группы АС-66
Ярома А. И.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы

Изучить архитектуру и программную модель сопроцессора, его регистры, форматы чисел и систему целочисленных и вещественных команд. Получить практические навыки вычисления разветвлённой функции средствами x87.

Задание. Вариант 12

Методическое пособие: страницы 316-318, таблица 2.2, вариант 12. Разделы 1 и 2 выполняются как одна работа с общим титульным листом. Разработать программу на ассемблере для вычисления функции с помощью регистров сопроцессора. Вычисление оформить отдельной функцией, каждую ветвь - отдельной внутренней процедурой. Ввод и вывод - через консоль. Проверить программу по всем условиям.

Во избежание неоднозначности результата, обозначенного в таблице как y, в программе используется имя r. Входные переменные остаются x и y.

Условие	Формула результата r
x - y > 2	(x*x - 4*y + 5) / (x*x + 2)
x - y < -4	(2*x + y) / (3*x*x + 4)
-4 <= x - y <= 2	x*x - 2*y + 3

Границы x - y = -4 и x - y = 2 включены в третью ветвь. Знаменатели $x^2 + 2$ и $3x^2 + 4$ положительны при всех вещественных x.

Среда выполнения

Windows, Microsoft Visual Studio 2026, MASM x86, платформа решения x86 (Win32 в проекте), набор инструментов v145. Поддерживаются Debug и Release. Исходный код программы полностью написан на ассемблере. Системная msvcrt.dll используется для числового ввода/вывода и преобразования текста в числа; формулы и выбор ветви выполняются командами x87.

Раздел 1. Целочисленные команды

EvaluateInteger принимает два 32-разрядных знаковых целых числа, адрес результата REAL8 и адрес номера ветви. FILD загружает целое в стек x87. FIADD, FISUB и FIMUL выполняют арифметику с целочисленными операндами памяти. FICOMP сравнивает разность с порогом; FNSTSW и SAHF передают результат сравнения условным переходам.

Три приватные процедуры IntAbove, IntBelow и IntMiddle реализуют соответствующие формулы. FDIVP делит числитель на знаменатель и освобождает один элемент стека. FSTP сохраняет окончательный результат как double. Целочисленные входы не означают усечение частного: при x = 3, y = 0 получается 14/11, то есть примерно 1,2727272727272727.

Раздел 2. Вещественные команды

EvaluateReal принимает адреса двух конечных чисел REAL8. Загрузка выполняется FLD, арифметика - FADD, FSUB, FMUL, FDIVP, сравнение - FCOMP. Ветви вынесены в RealAbove, RealBelow и RealMiddle. NaN и бесконечности отклоняются; не представимый конечным double результат вызывает сообщение об ошибке.

Обе функции используют расширенные регистры x87 с 64-разрядной мантиссой и округлением к ближайшему. Управляющее слово вызывающего кода сохраняется и восстанавливается. Стек x87 после вызова пуст, как и до него. Промежуточные квадраты не вычисляются в 32-разрядных регистрах общего назначения.

Порядок работы программы

Пользователь выбирает раздел 1 или 2 и вводит x и y отдельными строками. ReadNumber проверяет полный ввод числа. В первом разделе допустим диапазон -2147483648..2147483647, во втором - конечные значения double. Дробная часть задаётся точкой, поддерживается экспоненциальная запись. Длина строки ввода числа - до 255 байт. Затем вычислительная функция выбирает ветвь и возвращает результат; программа печатает входы, формулу и результат с 17 значащими цифрами. При недопустимом вводе или численной ошибке расчёт не выводится.

Текст программы

Вычислительные функции и шесть приватных процедур ветвей: variant12.asm

```
 1  .686
 2  .model flat, stdcall
 3  option casemap:none
 4  .const
 5  iTwo SDWORD 2
 6  iThree SDWORD 3
 7  iFour SDWORD 4
 8  iFive SDWORD 5
 9  iMinusFour SDWORD -4
10  rTwo REAL8 2.0
11  rThree REAL8 3.0
12  rFour REAL8 4.0
13  rFive REAL8 5.0
14  rMinusFour REAL8 -4.0
15  .code
16  ; Private branch procedures. Each leaves one value in ST(0).
17  ; Part 1 uses integer operands through FILD and FI arithmetic.
18  IntAbove PROC PRIVATE x:SDWORD, y:SDWORD
19      fild x
20      fimul x
21      fisub y
22      fisub y
23      fisub y
24      fisub y
25      fiadd iFive
26      fild x
27      fimul x
28      fiadd iTwo
29      fdivp st(1), st(0)
30      ret
31  IntAbove ENDP
32  IntBelow PROC PRIVATE x:SDWORD, y:SDWORD
33      fild x
34      fiadd x
35      fiadd y
36      fild x
37      fimul x
38      fimul iThree
39      fiadd iFour
40      fdivp st(1), st(0)
41      ret
42  IntBelow ENDP
43  IntMiddle PROC PRIVATE x:SDWORD, y:SDWORD
44      fild x
45      fimul x
46      fisub y
47      fisub y
48      fiadd iThree
49      ret
50  IntMiddle ENDP
51  ; Part 2 uses real memory operands through FLD and F arithmetic.
52  RealAbove PROC PRIVATE x:PTR REAL8, y:PTR REAL8
53      mov ecx, x
54      mov edx, y
55      fld REAL8 PTR [ecx]
56      fmul REAL8 PTR [ecx]
57      fld REAL8 PTR [edx]
58      fmul rFour
59      fsubp st(1), st(0)
60      fadd rFive
61      fld REAL8 PTR [ecx]
62      fmul REAL8 PTR [ecx]
63      fadd rTwo
64      fdivp st(1), st(0)
65      ret
66  RealAbove ENDP
67  RealBelow PROC PRIVATE x:PTR REAL8, y:PTR REAL8
68      mov ecx, x
69      mov edx, y
70      fld REAL8 PTR [ecx]
71      fmul rTwo
72      fadd REAL8 PTR [edx]
```

```
 73      fld REAL8 PTR [ecx]
 74      fmul REAL8 PTR [ecx]
 75      fmul rThree
 76      fadd rFour
 77      fdivp st(1), st(0)
 78      ret
 79  RealBelow ENDP
 80  RealMiddle PROC PRIVATE x:PTR REAL8, y:PTR REAL8
 81      mov ecx, x
 82      mov edx, y
 83      fld REAL8 PTR [ecx]
 84      fmul REAL8 PTR [ecx]
 85      fld REAL8 PTR [edx]
 86      fmul rTwo
 87      fsubp st(1), st(0)
 88      fadd rThree
 89      ret
 90  RealMiddle ENDP
 91  ; ABI: stdcall. Result and branch are output pointers; EAX=0 success,
 92  ; EAX=1 nonfinite input/output. Branch 1: d>2, 2: d<-4, 3: middle.
 93  ; Caller supplies valid pointers and an empty x87 stack (normal x86 ABI).
 94  ; Control word is restored; x87 stack is empty on return.
 95  EvaluateInteger PROC x:SDWORD, y:SDWORD, result:PTR REAL8, branch:PTR DWORD
 96      LOCAL oldCW:WORD, workCW:WORD
 97      fnstcw oldCW
 98      mov ax, oldCW
 99      and ax, 0F0FFh
100      or ax, 033Fh
101      mov workCW, ax
102      fldcw workCW
103      fild x
104      fisub y
105      ficomp iTwo
106      fnstsw ax
107      sahf
108      ja int_above
109      fild x
110      fisub y
111      ficomp iMinusFour
112      fnstsw ax
113      sahf
114      jb int_below
115      invoke IntMiddle, x, y
116      mov eax, 3
117      jmp int_finish
118  int_above:
119      invoke IntAbove, x, y
120      mov eax, 1
121      jmp int_finish
122  int_below:
123      invoke IntBelow, x, y
124      mov eax, 2
125  int_finish:
126      mov edx, branch
127      mov [edx], eax
128      mov edx, result
129      fstp REAL8 PTR [edx]
130      fldcw oldCW
131      xor eax, eax
132      ret
133  EvaluateInteger ENDP
134  EvaluateReal PROC x:PTR REAL8, y:PTR REAL8, result:PTR REAL8, branch:PTR DWORD
135      LOCAL oldCW:WORD, workCW:WORD
136      mov ecx, x
137      mov edx, y
138      mov eax, [ecx + 4]
139      and eax, 7FF00000h
140      cmp eax, 7FF00000h
141      je real_invalid
142      mov eax, [edx + 4]
143      and eax, 7FF00000h
144      cmp eax, 7FF00000h
145      je real_invalid
146      fnstcw oldCW
147      mov ax, oldCW
148      and ax, 0F0FFh
149      or ax, 033Fh
150      mov workCW, ax
```

```
151      fldcw workCW
152      fld REAL8 PTR [ecx]
153      fsub REAL8 PTR [edx]
154      fcomp rTwo
155      fnstsw ax
156      sahf
157      ja real_above
158      fld REAL8 PTR [ecx]
159      fsub REAL8 PTR [edx]
160      fcomp rMinusFour
161      fnstsw ax
162      sahf
163      jb real_below
164      invoke RealMiddle, x, y
165      mov eax, 3
166      jmp real_finish
167  real_above:
168      invoke RealAbove, x, y
169      mov eax, 1
170      jmp real_finish
171  real_below:
172      invoke RealBelow, x, y
173      mov eax, 2
174  real_finish:
175      mov edx, branch
176      mov [edx], eax
177      mov edx, result
178      fstp REAL8 PTR [edx]
179      fldcw oldCW
180      mov eax, [edx + 4]
181      and eax, 7FF00000h
182      cmp eax, 7FF00000h
183      je real_invalid
184      xor eax, eax
185      ret
186  real_invalid:
187      mov eax, 1
188      ret
189  EvaluateReal ENDP
190  END
```

Выбор раздела, ввод аргументов и вывод результата: main.asm

```
  1  ; Lab 4, variant 12: both sections of the x87 assignment, pages 316-318.
  2  .686
  3  .model flat, stdcall
  4  option casemap:none
  5  include numeric_io.inc
  6  SetConsoleCP PROTO :DWORD
  7  SetConsoleOutputCP PROTO :DWORD
  8  exit PROTO C :SDWORD
  9  .const
 10  header BYTE 'АПО ЭВМиС. Лабораторная работа №4',13,10
 11         BYTE 'Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87',13,10
 12         BYTE '1 - целочисленные команды; 2 - вещественные команды.',13,10
 13         BYTE 'Каждое число вводите на отдельной строке. Дробная часть: через точку.',13,10
 14         BYTE 'Выберите раздел (1/2): ',0
 15  promptX BYTE 'Введите x: ',0
 16  promptY BYTE 'Введите y: ',0
 17  intValues BYTE 13,10,'Раздел 1: целочисленные команды x87.',13,10
 18            BYTE 'x = %d; y = %d',13,10,0
 19  realValues BYTE 13,10,'Раздел 2: вещественные команды x87.',13,10
 20             BYTE 'x = %.17g; y = %.17g',13,10,0
 21  branchOne BYTE 'Ветвь 1: x - y > 2; r = (x*x - 4*y + 5) / (x*x + 2)',13,10,0
 22  branchTwo BYTE 'Ветвь 2: x - y < -4; r = (2*x + y) / (3*x*x + 4)',13,10,0
 23  branchThree BYTE 'Ветвь 3: -4 <= x - y <= 2; r = x*x - 2*y + 3',13,10,0
 24  resultFormat BYTE 'Результат: %.17g',13,10,0
 25  inputError BYTE 13,10,'Ошибка: некорректное число, диапазон или раздел.',13,10
 26             BYTE 'Целые: -2147483648..2147483647. Вещественные: конечные double.',13,10,0
 27  numericError BYTE 'Ошибка: результат не представим конечным double.',13,10,0
 28  .code
 29  ProgramMain PROC
 30      LOCAL mode:DWORD, xInt:SDWORD, yInt:SDWORD
 31      LOCAL xReal:REAL8, yReal:REAL8, result:REAL8, branch:DWORD
 32      invoke SetConsoleCP, 65001
 33      invoke SetConsoleOutputCP, 65001
 34      invoke printf, ADDR header
```

```
    invoke ReadNumber, 1, ADDR mode
    test eax, eax
    jnz input_bad
    cmp mode, 1
    je integer_input
    cmp mode, 2
    jne input_bad
    invoke printf, ADDR promptX
    invoke ReadNumber, 2, ADDR xReal
    test eax, eax
    jnz input_bad
    invoke printf, ADDR promptY
    invoke ReadNumber, 2, ADDR yReal
    test eax, eax
    jnz input_bad
    invoke printf, ADDR realValues, xReal, yReal
    invoke EvaluateReal, ADDR xReal, ADDR yReal, ADDR result, ADDR branch
    jmp checked
integer_input:
    invoke printf, ADDR promptX
    invoke ReadNumber, 1, ADDR xInt
    test eax, eax
    jnz input_bad
    invoke printf, ADDR promptY
    invoke ReadNumber, 1, ADDR yInt
    test eax, eax
    jnz input_bad
    invoke printf, ADDR intValues, xInt, yInt
    invoke EvaluateInteger, xInt, yInt, ADDR result, ADDR branch
checked:
    test eax, eax
    jnz numeric_bad
    cmp branch, 1
    je print_one
    cmp branch, 2
    je print_two
    invoke printf, ADDR branchThree
    jmp print_result
print_one:
    invoke printf, ADDR branchOne
    jmp print_result
print_two:
    invoke printf, ADDR branchTwo
print_result:
    invoke printf, ADDR resultFormat, result
    xor eax, eax
    ret
input_bad:
    invoke printf, ADDR inputError
    mov eax, 1
    ret
numeric_bad:
    invoke printf, ADDR numericError
    mov eax, 1
    ret
ProgramMain ENDP
start:
    finit
    invoke ProgramMain
    invoke exit, eax
END start
```

Объявления интерфейсов: numeric_io.inc

```
; MASM declarations. CRT is used for numeric parsing and printing only.
printf PROTO C :PTR BYTE, :VARARG
fflush PROTO C :DWORD
ReadNumber PROTO :DWORD, :DWORD
EvaluateInteger PROTO :SDWORD, :SDWORD, :PTR REAL8, :PTR DWORD
EvaluateReal PROTO :PTR REAL8, :PTR REAL8, :PTR REAL8, :PTR DWORD
```

Разбор и проверка числового ввода: numeric_io.asm

```
.686
.model flat, stdcall
option casemap:none
```

```
include numeric_io.inc
getchar PROTO C
strtol PROTO C :PTR BYTE, :PTR DWORD, :SDWORD
strtod PROTO C :PTR BYTE, :PTR DWORD
_errno PROTO C
.code
; Read one complete line. kind=1 -> signed DWORD, kind=2 -> REAL8.
; EAX=0 success, 1 invalid input/EOF/range/line too long. Max 255 bytes.
; The full line is consumed before validation; trailing junk is rejected.
ReadNumber PROC USES esi edi ebx, kind:DWORD, destination:DWORD
    LOCAL buffer[256]:BYTE, endPointer:DWORD, overflow:DWORD
    lea esi, buffer
    xor ebx, ebx
    mov overflow, 0
    invoke fflush, 0
read_loop:
    invoke getchar
    cmp eax, -1
    je line_end
    cmp eax, 10
    je line_end
    test eax, eax
    jz too_long
    cmp ebx, 255
    jae too_long
    mov [esi + ebx], al
    inc ebx
    jmp read_loop
too_long:
    mov overflow, 1
    jmp read_loop
line_end:
    mov BYTE PTR [esi + ebx], 0
    cmp overflow, 0
    jne invalid
    test ebx, ebx
    jz invalid
    invoke _errno
    mov DWORD PTR [eax], 0
    mov edi, destination
    cmp kind, 1
    jne parse_real
    invoke strtol, esi, ADDR endPointer, 10
    mov [edi], eax
    invoke _errno
    cmp DWORD PTR [eax], 0
    jne invalid
    jmp check_tail
parse_real:
    invoke strtod, esi, ADDR endPointer
    fstp REAL8 PTR [edi]
    mov eax, [edi + 4]
    and eax, 7FF00000h
    cmp eax, 7FF00000h
    je invalid
    invoke _errno
    cmp DWORD PTR [eax], 0
    je check_tail
    ; Accept representable subnormals; reject underflow to zero.
    mov eax, [edi + 4]
    and eax, 7FFFFFFFh
    or eax, [edi]
    jz invalid
check_tail:
    mov edx, endPointer
    cmp edx, esi
    je invalid
tail_loop:
    mov al, [edx]
    test al, al
    jz valid
    cmp al, 32
    je tail_next
    cmp al, 9
    jb invalid
    cmp al, 13
    ja invalid
tail_next:
```

```
82      inc edx
83      jmp tail_loop
84  valid:
85      xor eax, eax
86      ret
87  invalid:
88      mov eax, 1
89      ret
90  ReadNumber ENDP
91  END
```

Объявление импортируемых функций системной библиотеки: io_imports.def

```
1  LIBRARY msvcrt.dll
2  EXPORTS
3      printf
4      fflush
5      getchar
6      strtol
7      strtod
8      _errno
9      exit
```

Результаты. Раздел 1

Снимки получены при запуске собранной программы Release / x86. Числа введены через консоль; программа сама печатает x, y, выбранную формулу и числовой результат.

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 1
Введите x: 3
Введите y: 0

Раздел 1: целочисленные команды x87.
x = 3; y = 0
Ветвь 1: x - y > 2; r = (x*x - 4*y + 5) / (x*x + 2)
Результат: 1.2727272727272727
```

Рисунок 1 - x = 3, y = 0. Первая ветвь: r = 14/11.

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 1
Введите x: 0
Введите y: 5

Раздел 1: целочисленные команды x87.
x = 0; y = 5
Ветвь 2: x - y < -4; r = (2*x + y) / (3*x*x + 4)
Результат: 1.25
```

Рисунок 2 - x = 0, y = 5. Вторая ветвь: r = 5/4 = 1,25.

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 1
Введите x: 2
Введите y: 0

Раздел 1: целочисленные команды x87.
x = 2; y = 0
Ветвь 3: -4 <= x - y <= 2; r = x*x - 2*y + 3
Результат: 7
```

Рисунок 3 - x = 2, y = 0. Граница входит в третью ветвь: r = 7.

Результаты. Раздел 2

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 2
Введите x: 2.5
Введите y: 0.25

Раздел 2: вещественные команды x87.
x = 2.5; y = 0.25
Ветвь 1: x - y > 2; r = (x*x - 4*y + 5) / (x*x + 2)
Результат: 1.2424242424242424
```

Рисунок 4 - x = 2,5, y = 0,25. Первая ветвь: r ≈ 1,2424242424242424.

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 2
Введите x: 0.25
Введите y: 5

Раздел 2: вещественные команды x87.
x = 0.25; y = 5
Ветвь 2: x - y < -4; r = (2*x + y) / (3*x*x + 4)
Результат: 1.3134328358208955
```

Рисунок 5 - x = 0,25, y = 5. Вторая ветвь: r ≈ 1,3134328358208955.

```
АПО ЭВМиС. Лабораторная работа №4
Ярома А. И. | АС-66 | Вариант 12 | MASM x86 / x87
1 - целочисленные команды; 2 - вещественные команды.
Каждое число вводите на отдельной строке. Дробная часть: через точку.
Выберите раздел (1/2): 2
Введите x: 0.5
Введите y: 1.25

Раздел 2: вещественные команды x87.
x = 0.5; y = 1.25
Ветвь 3: -4 <= x - y <= 2; r = x*x - 2*y + 3
Результат: 0.75
```

Рисунок 6 - x = 0,5, y = 1,25. Третья ветвь: r = 0,75.

Проверка граничных условий

Раздел / входы	Ветвь	Ожидаемый результат
Оба: x = 2, y = 0	3	7
Оба: x = 0, y = 4	3	-5
Целые: x = 3, y = 0	1	14/11
Целые: x = 0, y = 5	2	1,25
Вещественные: 2,5 и 0,25	1	41/33
Вещественные: 0,25 и 5	2	88/67
Вещественные: 0,5 и 1,25	3	0,75
NaN, Inf, выход за диапазон	Ошибка	Результат не печатается

Проведено 12 024 проверки вычислительных функций: фиксированные примеры, случайные пары, крайние signed int, границы и соседние double, восстановление управляющего слова, повторные вызовы, нечисловые входы и переполнение результата. Ошибок не обнаружено.

В каждой конфигурации Debug и Release выполнено 369 проверок программы через стандартный ввод/вывод. Эталон вычислялся независимой рациональной арифметикой Python Fraction. Дополнительно по шесть проверок в каждой конфигурации выполнено вводом с клавиатуры в настоящей консоли Windows. Итого - 12 774 успешные проверки.

Вывод

Реализованы оба раздела работы: одна функция использует целочисленные операнды x87, другая - вещественные. Каждой ветви соответствует отдельная приватная процедура. Проверки подтвердили правильный выбор ветвей и включение обеих границ в средний интервал. Расширенный формат регистров сопроцессора позволяет сохранить дробный результат деления и избежать переполнения 32-разрядного целого при вычислении квадрата.

Источник задания

Лекционный материал и лабораторный курс по дисциплине АПО ЭВМ и С. Часть 2 «Архитектура сопроцессора», стр. 316-318, таблица 2.2, вариант 12.